

INTEGRATED POLAR SCIENCE OUTREACH, KNOWLEDGE REPOSITORY & MEDIA DISSEMINATION PORTAL

Complete Brainstorm → Product Architecture → Pages → Features → APIs → Database → AI Service → Implementation Guidance

Problem Statement ID	26063
Problem Statement	Integrated Polar Science Outreach, Knowledge Repository and Media Dissemination Portal
Organization	Ministry of Earth Sciences (MoES)
Department	National Centre for Polar and Ocean Research (NCPOR)
Category	Software
Theme	Smart Education

0. Original Product Brainstorm — Preserve the Full Intent

The portal should be a comprehensive polar-science outreach and knowledge system rather than a generic document website. It should archive expedition reports, scientific datasets, research, publications, photographs, videos and institutional activities while also turning that knowledge into searchable, understandable and reusable public-facing content.

- Home must remain clutter-free and act as a discovery/intelligence dashboard rather than exposing every feature at once.
- Sidebar must expose Social Media Posts, Research Connection, Institutional Activities and AI as clear sections.
- Semantic Search should be prominent in the top navigation/header because it is a core USP.
- AI Data Trends should appear on Home as a visual intelligence section.
- Knowledge Graph should have a Home preview and a dedicated full-page experience.
- Notifications should be a top-right icon and include newly generated posts, newly added reports, data changes, new publications, new datasets and institutional activities.
- Login and Signup should be visible when logged out; after login they should become a profile icon.
- The portal should have a complete Polar Knowledge Repository covering expedition reports, scientific datasets, research, publications, photographs, reports, conferences, seminars, workshops and awareness events.
- Search and filtering must work across repository content.
- Reports should support AI summarization.
- The platform should generate social media posts from scientific content.
- USP features are Knowledge Graph, Research Connection and Semantic Search.
- Recent uploaded data should be visible both as a Home feature and through notifications.
- The Smart Education theme should be represented through a student-friendly learning experience and simplified scientific explanations.

1. Product Goals

- Centralize polar scientific knowledge in one searchable repository.
- Make scientific content discoverable through both conventional and semantic search.
- Connect researchers, institutions, expeditions, publications, datasets and locations.
- Use AI to summarize, classify, connect and transform scientific content without removing human review.
- Turn repository content into outreach material for websites and social platforms.
- Provide trend intelligence from accumulated repository data.
- Make complex polar science accessible to students and the public.
- Maintain scientific provenance, source attribution and content/version history.
- Provide an administration layer for ingestion, moderation, approvals and analytics.

2. Target User Groups

- Students: discover simple educational polar-science resources, reports, videos and quizzes.
- Researchers: search publications, datasets, reports, expeditions, researchers and institutions.
- Scientific institutions: publish and organize reports, datasets, research and activities.
- Educators: find reliable content and convert scientific material into teaching resources.
- General public: explore polar science through simplified explanations, media and outreach.
- Content/Admin teams: manage repository records, uploads, metadata, AI-generated content and moderation.
- Outreach/Social teams: generate, review, schedule and publish platform-specific scientific communication.

3. Global Information Architecture

POLAR PORTAL ■■■ Home ■■■ Explore / Semantic Search ■■■ Expedition Explorer ■■■ Knowledge Repository ■ ■■■ Reports ■ ■■■ Scientific Datasets ■ ■■■ Research ■ ■■■ Publications ■ ■■■ Photographs ■ ■■■ Videos ■■■ Outreach ■ ■■■ Social Media Studio ■ ■■■ Institutional Activities ■ ■■■ Conferences ■ ■■■ Seminars ■ ■■■ Workshops ■ ■■■ Awareness Events ■■■ AI / Intelligence ■ ■■■ AI Research Assistant ■ ■■■ Document Analyzer ■ ■■■ Polar Science Trends ■ ■■■ Knowledge Graph ■ ■■■ Research Connections ■■■ Education ■ ■■■ Polar Education ■ ■■■ Learning Topics ■■■ User ■ ■■■ Profile ■ ■■■ Notifications ■ ■■■ Saved Content ■ ■■■ History ■■■ Settings ■■■ Admin ■■■ Dashboard ■■■ Content Management ■■■ Dataset Management ■■■ User Management ■■■ AI Approval ■■■ Analytics

4. Home Page — Detailed Brainstorm

- Header: logo, primary navigation, prominent semantic search, notification icon, login/signup or profile icon.
- Hero: short mission statement about polar science knowledge and outreach plus a large natural-language search box.
- Search examples: 'Find Antarctic sea-ice research', 'Show reports from recent expeditions', 'Researchers working on polar biodiversity'.
- Quick Access: Expeditions, Reports, Datasets, Research, Publications, Photographs, Videos, Conferences, Seminars, Workshops and Awareness Events.
- Recent Uploads: latest repository additions with content type, title, institution, date and actions.
- AI Data Trends: research growth, publication activity, expedition activity, dataset growth, frequent topics and emerging areas.
- Knowledge Graph Preview: a compact interactive graph showing relationships among researchers, institutions, expeditions, publications and datasets.
- Featured Research: curated or algorithmically relevant research and publications.
- Upcoming Activities: next conferences, seminars, workshops and outreach programs.
- Featured Expedition: visual card with location, dates, objectives and related resources.
- Home should not contain full admin controls, complete graph controls or every repository filter; those belong on dedicated pages.

5. Sidebar Design

- MAIN: Home, Explore, Expedition Explorer.
- KNOWLEDGE: Knowledge Repository, Reports, Scientific Datasets, Research, Publications, Photographs, Videos.
- OUTREACH: Social Media Studio, Institutional Activities, Conferences, Seminars, Workshops, Awareness Events.
- INTELLIGENCE: AI Polar Intelligence, Knowledge Graph, Research Connections.
- USER: Saved Content, History, Profile.
- The sidebar should be collapsible so the scientific content remains the dominant visual area.

6. Page-by-Page Design Specification

6.1 Home

- Discovery dashboard with search, quick categories, recent uploads, trends, graph preview and activities.

6.2 Explore / Semantic Search

- Search results page with natural-language search, filters, sorting, relevance scores, related queries and result previews.

6.3 Expedition Explorer

- Interactive Arctic/Antarctic map plus expedition timeline and location-based exploration.

6.4 Knowledge Repository

- Central content browser with tabs/categories for reports, datasets, research, publications, media and events.

6.5 Reports

- Report library with filters, document metadata, AI summary and related resources.

6.6 Report Detail

- Original report metadata, document viewer, AI-generated summary, key findings, related research, datasets, expedition and citation/provenance.

6.7 Scientific Datasets

- Dataset catalog with metadata, variables, source, collection location/date, version, license, download and related publications.

6.8 Dataset Detail

- Dataset information, preview/visualization, versions, change history, related research, download and source information.

6.9 Research

- Research project browser with domain, researcher, institution, expedition, publication and dataset relationships.

6.10 Publications

- Scientific publication library with authors, journal, DOI, year, domain and related resources.

6.11 Photographs

- Searchable gallery with location, expedition, date, creator and research association.

6.12 Videos

- Video library for expeditions, research, interviews, education and institutional outreach.

6.13 Social Media Studio

- Select source content, platform, audience/tone and language; generate draft; edit; approve; schedule/publish.

6.14 Institutional Activities

- Activity browser covering conferences, seminars, workshops, awareness programs, training and exhibitions.

6.15 Event Detail

- Date, venue, speakers, institution, description, registration, media and related research.

6.16 AI Polar Intelligence

- Central AI hub containing research assistant, document analyzer, metadata generation, trends and outreach generation.

6.17 AI Research Assistant

- Repository-grounded chat that answers questions using retrieved scientific sources.

6.18 AI Document Analyzer

- Summarizes documents and extracts findings, methodology, entities, keywords and related content.

6.19 Polar Science Intelligence

- Charts and AI insights for research trends, publication trends, expedition activity, datasets and emerging topics.

6.20 Knowledge Graph

- Full-screen graph exploration with filters, node detail and relationship traversal.

6.21 Research Connections

- Discover researchers, institutions, publications, datasets and topics connected to a research interest.

6.22 Notifications

- Full notification inbox with unread/read states and categories.

6.23 Profile

- User identity, institution, research interests, saved items, followed topics and activity.

6.24 Saved Content

- Personal library of saved reports, datasets, publications, expeditions, researchers and media.

6.25 History

- Recently viewed resources and recent searches.

6.26 Polar Education

- Student-friendly learning hub with basic polar science, climate, glaciers, biodiversity, expeditions and remote sensing.

6.27 Learning Topic Detail

- Simplified explanation, visuals, videos, related research, glossary and quiz.

6.28 Login / Signup / Password Reset

- Secure authentication flows with simple, clean screens.

6.29 Settings

- Account, notification preferences, language, appearance, privacy and preferences.

6.30 Admin Dashboard

- Operational overview of repository size, users, pending content, AI approvals, usage and growth.

6.31 Content Management

- Admin CRUD, categorization, publishing, moderation and metadata correction.

6.32 Dataset Management

- Dataset upload, metadata, versions, updates and change history.

6.33 User Management

- User, researcher and institutional accounts, roles and permissions.

6.34 AI Approval

- Review AI-generated summaries, metadata and social posts before publication.

6.35 Admin Analytics

- Search behavior, popular content, repository growth, dataset usage, AI usage and engagement.

7. Repository Features — Full Brainstorm

- Expedition reports with expedition metadata and related resources.
- Scientific datasets with structured metadata and versions.
- Research projects with researchers, institutions and outputs.
- Publications with DOI, authors and related datasets.
- Technical and scientific reports.
- Photograph archive.
- Video archive.
- Conferences.
- Seminars.
- Workshops.
- Awareness events.
- Institutional announcements and outreach programs.
- Cross-link every content type wherever a relationship exists.
- Allow users to save and share content.
- Expose source/provenance and last-updated information.

8. Search & Filtering

- Global semantic search should search across reports, datasets, publications, research, expeditions and media metadata.
- Natural-language queries should retrieve conceptually related content even when exact keywords differ.
- Filters: content type, Arctic/Antarctic, year, expedition, institution, research domain, author/researcher and event type.
- Sorting: relevance, newest, oldest and optionally popularity/view count.
- Search results should show title, type, short snippet, relevance score, source and quick actions.
- Search suggestions should be generated from indexed titles, entities and common terms.
- Recent searches should be stored for authenticated users.
- Search results should link directly to source documents/resources.

9. AI Data Trends

- Research activity trend by year/month.
- Publication growth over time.
- Dataset additions and updates.
- Expedition activity by region/year.
- Research domain distribution.
- Frequent scientific topics and keywords.
- Emerging topics detected from recent content.
- Arctic versus Antarctic content distribution.
- Institutional research activity distribution.
- AI-generated narrative insight explaining observed repository patterns.
- Trend outputs should be treated as analytical summaries, with source data and calculation period visible.

10. Knowledge Graph — Full Brainstorm

- Entities: Researcher, Institution, Expedition, Location, Research Project, Report, Dataset, Publication, Media, Topic and Event.
- Relationships: works-at, authored, participated-in, studies, produced, uses-dataset, related-to, located-at and organized-by.
- Node click should show metadata and connected resources.
- Graph filters should include entity type, region, year and research domain.
- Graph search should allow a researcher, institution, topic or expedition to become the focus node.
- For the MVP, construct nodes and edges from PostgreSQL relationships.
- Neo4j can be introduced later if graph traversal becomes a genuine scale requirement.

11. Research Connection — Full Brainstorm

- Input a research topic or select an existing research record.
- Find related researchers.
- Find related institutions.
- Find related publications.
- Find related datasets.
- Find related expeditions.
- Find related topics.
- Show why each connection exists, such as shared topic, publication, dataset or expedition.
- Allow users to open the underlying source records rather than presenting unsupported AI-only connections.

12. Report Summarization

- Generate executive summary.
- Extract key findings.
- Extract methodology.
- Extract important measurements/numbers where reliably available.
- Extract entities such as researchers, institutions, locations and expeditions.
- Generate keywords and research domains.
- Show related repository content.
- Allow users to ask questions about the report.
- Store summary provenance including source document, generation timestamp and model information.

13. Social Media Generation

- Platforms: LinkedIn, Instagram, X, Facebook and YouTube where integrations are available.
- Inputs: source report/publication/dataset/event, platform, tone, audience, language and desired length.

- Outputs: caption/post, hashtags, short description, CTA and optional visual suggestion.
- Workflow must be AI Generate → Draft → Human Review → Approve → Schedule/Publish.
- Maintain generated content history and approval status.
- Allow editing before publication.
- Keep public-facing AI output attributable to the source scientific record.

14. Notifications — Full Requirement

- New report uploaded.
- New dataset added.
- Dataset updated or version changed.
- New publication added.
- New expedition documented.
- New photograph/video collection added.
- New AI insight generated.
- AI summary completed.
- AI social post generated.
- New conference, seminar, workshop or awareness event added.
- Updates to followed topics/researchers where that preference exists.
- Top-right notification icon should show unread count.
- Notification center should support mark-read and mark-all-read.

15. Smart Education

- Create simple educational modules around Arctic, Antarctica, climate, glaciers, biodiversity, oceanography, remote sensing and expeditions.
- Convert technical concepts into student-friendly explanations.
- Use visuals, videos, maps and infographics.
- Connect learning content to original scientific reports and publications.
- Add quizzes for engagement.
- Add glossary for scientific terminology.
- Allow educators/students to navigate from simple content to primary scientific sources.

16. Data Provenance & Trust

- Every scientific record should expose source institution.
- Show authors/creators where available.
- Show upload and last-update timestamps.
- Show dataset version when applicable.
- Show license/access conditions.
- Show collection/expedition context.
- Show source document or storage reference.
- AI-generated outputs should retain source record IDs and generation metadata.
- Keep human approval for public-facing generated content.

17. Database Decision — PostgreSQL

- Primary database: PostgreSQL.
- Reason: the system has strong structured relationships among users, researchers, institutions, expeditions, reports, datasets, publications, events and media.
- Use normalized relational tables and explicit relationship tables where many-to-many connections exist.
- Use PostgreSQL constraints, indexes and transactions for reliable scientific metadata and administrative workflows.

- Use pgvector for embeddings and semantic similarity search in the initial system.
- Do not introduce MongoDB simply because some AI content is semi-structured; JSON/JSONB fields in PostgreSQL can handle flexible metadata where appropriate.
- Do not introduce Neo4j at the MVP stage unless graph complexity justifies it.
- Dataset versioning is separate from API versioning and should be supported if datasets evolve.

18. File Storage

- Store PDFs, videos, photographs and large datasets in object storage rather than PostgreSQL binary columns.
- Possible storage: Amazon S3, Cloudflare R2, Supabase Storage, Azure Blob or MinIO.
- PostgreSQL stores file metadata, object key/URL, MIME type, size, source, owner, timestamps and relationships.
- Use signed URLs or controlled download endpoints where access restrictions apply.

19. AI Service Decision

- Create a separate logical Python FastAPI AI service.
- Do not make it an unnecessarily complex microservice ecosystem for the MVP.
- Node/Express remains the application backend and API gateway.
- Python handles LLM orchestration, embeddings, RAG, document processing, metadata extraction, entity extraction and trend analysis.
- Frontend never calls the Python AI service directly.
- Recommended flow: React → Node/Express → Python FastAPI AI Service → LLM/Embeddings/Vector Layer.
- This separation allows the AI stack to evolve independently without exposing provider credentials or internal AI endpoints to the browser.

20. AI Service Internal Structure

```
ai-service/
├── app.py
├── routes/
│   ├── summarize.py
│   ├── ask.py
│   ├── embeddings.py
│   ├── trends.py
│   ├── knowledge_graph.py
│   ├── social.py
│   └── services/
│       ├── llm.py
│       ├── embeddings.py
│       └── rag.py
├── document_processor.py
├── trend_engine.py
├── schemas/
└── requirements.txt
```

21. AI / RAG Processing Pipeline

```
Document Upload ↓ Object Storage ↓ PostgreSQL Metadata Record ↓ Processing Job ↓ PDF/Text/OCR Extraction ↓ Cleaning + Chunking ↓ Embeddings ↓ PostgreSQL + pgvector ↓ Entity / Relationship Extraction ↓ Summary / Metadata / Knowledge Graph Updates ↓ Notification ↓ Searchable + AI-usable Repository
```

- For large documents, process asynchronously rather than blocking an HTTP request.
- Use job IDs and status endpoints for processing progress.
- The AI layer should cite or return source resource IDs so the frontend can show grounded references.

22. Asynchronous AI Job Pattern

- POST /api/ai/summarize should create a job for large documents instead of waiting for completion.
- Return HTTP 202 with a jobId and processing status.
- GET /api/ai/jobs/:jobId should return queued, processing, completed or failed status.
- On completion, store the result and create a notification.
- Redis + BullMQ can be introduced for reliable background job processing when required.

23. Backend API Convention

- Use unversioned API links as requested: /api/reports, not /api/v1/reports.
- Use plural REST resource names.
- Use :id for individual resource paths.
- Use query parameters for filtering, sorting, pagination and search options.
- Do not create separate routes for every possible filter.
- Keep authentication and authorization in middleware.
- Keep validation at API boundaries.
- Use consistent error response structure.
- Paginate repository lists and search results.

24. Complete API Endpoint Map

24.1 Authentication

- POST /api/auth/signup — create account.
- POST /api/auth/login — login.
- POST /api/auth/logout — logout.
- POST /api/auth/refresh — refresh authentication.
- GET /api/auth/me — current user.
- POST /api/auth/forgot-password — start password reset.
- POST /api/auth/reset-password — complete password reset.

24.2 Users

- GET /api/users/me — current profile.
- PATCH /api/users/me — update current profile.
- GET /api/users/me/saved — saved resources.
- POST /api/users/me/saved — save resource.
- DELETE /api/users/me/saved/:contentId — unsave resource.
- GET /api/users/me/history — browsing/search history.
- DELETE /api/users/me/history — clear history.
- GET /api/users/:id — public profile where permitted.

24.3 Reports

- GET /api/reports — list reports.
- GET /api/reports/:id — report detail.
- POST /api/reports — create/upload report record.
- PATCH /api/reports/:id — update report.
- DELETE /api/reports/:id — delete report.
- GET /api/reports?year=2026®ion;=antarctica&institutionId;=... — filtered reports.

24.4 Datasets

- GET /api/datasets — list datasets.
- GET /api/datasets/:id — dataset detail.
- POST /api/datasets — create dataset record/upload metadata.
- PATCH /api/datasets/:id — update dataset.
- DELETE /api/datasets/:id — delete dataset.
- GET /api/datasets/:id/versions — version history.
- GET /api/datasets/:id/download — controlled download.
- GET /api/datasets/:id/related — related resources.

24.5 Publications

- GET /api/publications — list publications.
- GET /api/publications/:id — publication detail.
- POST /api/publications — create publication.
- PATCH /api/publications/:id — update publication.
- DELETE /api/publications/:id — delete publication.
- GET /api/publications/:id/related — related resources.

24.6 Research

- GET /api/research — list research projects.
- GET /api/research/:id — research detail.
- POST /api/research — create research.

- PATCH /api/research/:id — update research.
- DELETE /api/research/:id — delete research.
- GET /api/research/:id/researchers — related researchers.
- GET /api/research/:id/publications — related publications.
- GET /api/research/:id/datasets — related datasets.

24.7 Expeditions

- GET /api/expeditions — list expeditions.
- GET /api/expeditions/:id — expedition detail.
- POST /api/expeditions — create expedition.
- PATCH /api/expeditions/:id — update expedition.
- DELETE /api/expeditions/:id — delete expedition.
- GET /api/expeditions/:id/reports — reports.
- GET /api/expeditions/:id/datasets — datasets.
- GET /api/expeditions/:id/media — media.
- GET /api/expeditions/:id/research — research.

24.8 Media

- GET /api/media — list media.
- GET /api/media/:id — media detail.
- POST /api/media — create media record.
- PATCH /api/media/:id — update media.
- DELETE /api/media/:id — delete media.
- GET /api/media?type=photo — photographs.
- GET /api/media?type=video — videos.

24.9 Institutions

- GET /api/institutions — list institutions.
- GET /api/institutions/:id — institution detail.
- POST /api/institutions — create institution.
- PATCH /api/institutions/:id — update institution.
- DELETE /api/institutions/:id — delete institution.
- GET /api/institutions/:id/research — research.
- GET /api/institutions/:id/researchers — researchers.
- GET /api/institutions/:id/activities — activities.

24.10 Researchers

- GET /api/researchers — list researchers.
- GET /api/researchers/:id — researcher detail.
- GET /api/researchers/:id/publications — publications.
- GET /api/researchers/:id/research — research.
- GET /api/researchers/:id/expeditions — expeditions.
- GET /api/researchers/:id/connections — research connections.

24.11 Activities

- GET /api/activities — list institutional activities.
- GET /api/activities/:id — activity detail.
- POST /api/activities — create activity.
- PATCH /api/activities/:id — update activity.
- DELETE /api/activities/:id — delete activity.

24.12 Search

- POST /api/search — semantic/global search.
- GET /api/search/suggestions?q=... — search suggestions.
- GET /api/search/recent — recent searches.

24.13 Knowledge Graph

- GET /api/knowledge-graph — graph nodes and edges.
- GET /api/knowledge-graph/node/:id — node detail.
- GET /api/knowledge-graph/node/:id/relationships — connected nodes/edges.

24.14 Research Connections

- GET /api/connections/researchers/:id — researcher connections.
- GET /api/connections/research — related research.
- GET /api/connections/institutions/:id — institution connections.
- POST /api/connections/discover — discover connections from a topic.

24.15 AI

- POST /api/ai/summarize — summarize document.
- POST /api/ai/ask — repository-grounded AI question.
- POST /api/ai/generate-social-post — generate social content.
- POST /api/ai/generate-metadata — generate/extract metadata.
- GET /api/ai/insights — AI insights.
- GET /api/ai/trends — AI/data trends.
- POST /api/ai/recommend — recommendations.
- GET /api/ai/jobs/:jobId — AI job status.

24.16 Social

- GET /api/social/posts — list posts.
- GET /api/social/posts/:id — post detail.
- POST /api/social/posts/generate — generate draft.
- PATCH /api/social/posts/:id — edit draft.
- POST /api/social/posts/:id/approve — approve.
- POST /api/social/posts/:id/schedule — schedule.
- POST /api/social/posts/:id/publish — publish.
- DELETE /api/social/posts/:id — delete.

24.17 Notifications

- GET /api/notifications — all notifications.
- GET /api/notifications/unread — unread notifications.
- PATCH /api/notifications/:id/read — mark one read.
- POST /api/notifications/read-all — mark all read.
- DELETE /api/notifications/:id — delete.

24.18 Education

- GET /api/education/topics — topics.
- GET /api/education/topics/:id — topic detail.
- GET /api/education/topics/:id/related — related scientific resources.
- GET /api/education/quizzes/:id — quiz.
- POST /api/education/quizzes/:id/submit — submit quiz.

24.19 Admin

- GET /api/admin/dashboard — admin summary.

- GET /api/admin/analytics — admin analytics.
- GET /api/admin/content/pending — moderation queue.
- POST /api/admin/content/:id/approve — approve content.
- POST /api/admin/content/:id/reject — reject content.
- GET /api/admin/users — user management.
- PATCH /api/admin/users/:id/role — role management.
- GET /api/admin/ai/pending — AI review queue.
- POST /api/admin/ai/:id/approve — approve AI content.
- POST /api/admin/ai/:id/reject — reject AI content.

25. Suggested PostgreSQL Data Model

users institutions researchers research_domains research_projects expeditions locations reports datasets
dataset_versions publications media activities topics notifications saved_items search_history social_posts ai_jobs
ai_outputs knowledge_entities knowledge_relationships embeddings roles / permissions

- Use foreign keys for core relationships.
- Use join tables for many-to-many relationships such as researcher_publications, researcher_expeditions, publication_datasets, expedition_reports and research_datasets.
- Use JSONB selectively for flexible metadata rather than turning the entire database into unstructured JSON.
- Index common filters such as year, region, institution, type and research domain.
- Use pgvector for embedding similarity search.

26. Example Core Relationships

Researcher ■■■works_at■■■> Institution Researcher ■■■participates_in■■■> Expedition Researcher ■■■authors■■■>
Publication Researcher ■■■works_on■■■> Research Research ■■■uses■■■> Dataset Expedition ■■■produces■■■> Report Expedition
■■■produces■■■> Dataset Expedition ■■■contains■■■> Media Publication ■■■references/relates_to■■■> Dataset Publication
■■■relates_to■■■> Research Activity ■■■organized_by■■■> Institution Report/Dataset/Publication ■■■has_topic■■■> Topic

27. Node Backend Structure

backend/ ■■■ src/ ■■■ config/ ■■■ middleware/ ■■■ utils/ ■■■ modules/ ■ ■■■ auth/ ■ ■■■ users/ ■ ■■■ reports/ ■
■■■ datasets/ ■ ■■■ publications/ ■ ■■■ research/ ■ ■■■ expeditions/ ■ ■■■ media/ ■ ■■■ institutions/ ■ ■■■
researchers/ ■ ■■■ activities/ ■ ■■■ search/ ■ ■■■ knowledgeGraph/ ■ ■■■ connections/ ■ ■■■ notifications/ ■ ■■■
social/ ■ ■■■ education/ ■ ■■■ admin/ ■ ■■■ ai/ ■ ■■■ ai.controller.js ■ ■■■ ai.service.js ■ ■■■ ai.routes.js ■■■
app.js ■■■ server.js

28. Frontend Architecture Guidance

- Create reusable layout components: AppShell, Sidebar, Header, NotificationBell, ProfileMenu and SearchBar.
- Create reusable content components: ReportCard, DatasetCard, PublicationCard, ExpeditionCard, ResearcherCard, InstitutionCard and MediaCard.
- Create reusable data states: Loading, EmptyState, ErrorState, Pagination and FilterBar.
- Use a consistent design system for typography, spacing, cards, badges, buttons, tables, charts and graph controls.
- Keep scientific content readable and avoid dashboard-style visual overload.
- Use progressive disclosure: show summary first, details on demand.
- Make important source/provenance information visible without dominating the design.

29. Recommended Tech Stack

- Frontend: React or Next.js.
- Main backend: Node.js + Express.
- Database: PostgreSQL.
- Vector search: pgvector.
- AI service: Python + FastAPI.
- LLM: external provider behind an internal abstraction layer.
- File storage: S3/R2/Supabase Storage/Azure Blob/MinIO.
- Background jobs: Redis + BullMQ when asynchronous workloads require it.

- Knowledge Graph visualization: React Flow or Cytoscape.js.
- Charts: a React-compatible charting library.
- Authentication: secure cookie/session strategy or short-lived access credentials with refresh mechanism.
- Deployment can use Docker so Node, Python AI service and supporting services are reproducible.

30. What Not to Overengineer Initially

- Do not use MongoDB + PostgreSQL together without a concrete requirement.
- Do not add Neo4j only because a Knowledge Graph exists; PostgreSQL relationships are sufficient for an MVP.
- Do not add a separate vector database when pgvector is sufficient for the initial scale.
- Do not expose the Python AI service directly to the browser.
- Do not make every AI operation synchronous.
- Do not allow generated social content to bypass human approval.
- Do not create API versions in URLs because the agreed convention is unversioned /api/resource paths.
- Do not store large PDFs/videos/images directly in PostgreSQL.

31. End-to-End Example — Uploading a Report

Admin uploads PDF ↓ Node receives upload + metadata ↓ File stored in object storage ↓ PostgreSQL report record created ↓ AI processing job created ↓ Python extracts text/OCR ↓ Text is chunked ↓ Embeddings generated ↓ Embeddings stored in pgvector ↓ AI extracts metadata/entities ↓ Summary generated ↓ Knowledge relationships updated ↓ Report becomes searchable ↓ Notification generated ↓ Report appears in Recent Uploads

32. End-to-End Example — Semantic Search

User: "Find research about Antarctic sea-ice changes" ↓ React → POST /api/search ↓ Node validates query + filters ↓ AI/search layer creates query embedding ↓ pgvector retrieves semantically similar chunks/resources ↓ Optional RAG retrieves supporting source content ↓ Results ranked ↓ Frontend displays reports/publications/datasets ↓ User opens original source

33. End-to-End Example — AI Social Post

User selects Report ↓ POST /api/social/posts/generate ↓ Node sends source metadata/content to AI service ↓ AI creates platform-specific draft ↓ Draft saved as social_post ↓ User edits ↓ POST /api/social/posts/:id/approve ↓ Schedule or publish ↓ Notification/history updated

34. End-to-End Example — Knowledge Graph

Researcher ↓ Institution ↓ Research Project ↓ Publication ↓ Dataset ↓ Expedition ↓ Location PostgreSQL relationships ↓ Graph API ↓ nodes + edges JSON ↓ React Flow / Cytoscape ↓ Interactive graph

35. MVP Development Order

- Step 1: Set up repository, Git workflow, environment variables and project structure.
- Step 2: Set up PostgreSQL schema and migrations.
- Step 3: Build authentication and role-based access.
- Step 4: Build repository CRUD for reports, datasets, publications, research and expeditions.
- Step 5: Add object storage and file metadata.
- Step 6: Build Home, Sidebar and repository pages.
- Step 7: Build semantic search with pgvector.
- Step 8: Add Python AI service and report summarization.
- Step 9: Add AI Research Assistant/RAG.
- Step 10: Add Knowledge Graph.
- Step 11: Add Research Connections.
- Step 12: Add AI Data Trends.
- Step 13: Add Social Media Studio and approval workflow.
- Step 14: Add notifications.
- Step 15: Add Education mode.
- Step 16: Build Admin moderation and analytics.

- Step 17: Add provenance/version history and production hardening.

36. Suggested Team Division

- Frontend Engineer: AppShell, Home, repository UI, search UI, graph UI, charts and responsive design.
- Backend Engineer: Express API, PostgreSQL schema, authentication, CRUD, permissions and notifications.
- AI Engineer: FastAPI service, embeddings, RAG, summarization, metadata extraction and trends.
- Data/Repository Engineer: ingestion pipeline, metadata quality, object storage, dataset versioning and provenance.
- Outreach/Admin Engineer: social generation workflow, institutional activities, moderation and analytics.
- For a smaller team, combine Backend + Data and AI + Outreach roles.

37. Jury / Demonstration Flow

- Start on Home and show the clean design and semantic search.
- Search a natural-language polar research question.
- Open a report and demonstrate AI summary with source references.
- Show related datasets/publications/researchers.
- Open Knowledge Graph and demonstrate the relationship view.
- Open Research Connections to discover connected researchers/institutions.
- Show AI Data Trends derived from repository content.
- Generate a social post from the same scientific report and show human approval.
- Show notification generated by a new report or completed AI job.
- Finish with Smart Education mode showing how complex research can be made accessible to students.
- Emphasize that AI assists discovery and content transformation while source data, provenance and human review remain central.

38. Final Architecture

[illegible]

39. Final Decisions to Lock

- Database: PostgreSQL.
- Vector search: pgvector.
- API URLs: unversioned /api/... paths.
- Main backend: Node.js + Express.
- AI processing: logically separate Python + FastAPI service.
- Frontend communicates with Node, not directly with AI service.
- Files: object storage, not PostgreSQL binary storage.
- Knowledge Graph: PostgreSQL relationships initially.
- AI workflow: source-grounded + human review for public-facing output.
- Notifications: centralized event/notification model.
- Home: clean discovery/intelligence page.
- Sidebar: organized into Main, Knowledge, Outreach, Intelligence and User.
- Smart Education: dedicated learning mode connected to primary scientific sources.

40. Final One-Line Product Definition

One-line pitch: A unified AI-assisted Polar Science Knowledge and Outreach Platform that connects expeditions, research, datasets, publications, researchers and institutions through semantic search, knowledge graphs, scientific intelligence and responsible AI-powered content dissemination.

Implementation note: This document intentionally preserves the full brainstorming and architectural reasoning instead of reducing the project to a short checklist; it is meant to be used as the team's working reference while designing, coding and presenting the platform.